

Working with Files

MODULE 2: FILE OPERATIONS, LINKS, & DATA PROCESSING

INDEX

- Categories of File Creation Methods
- Categorisation of File Viewing Methods
- Categorisation of Hidden Methods
- Categorisation of Copying Methods
- Categorisation of Moving and Renaming Methods

Categories of File Creation Methods

- Command-Line Tools (Non-Interactive)
  - touch
  - cat
  - printf
  - mv
- Text Editors (Interactive)
  - vi/vim
  - nano
  - gedit

⚙ Command-Line Tools (Non-Interactive)

- **touch** – Creates a completely blank file instantaneously without opening any interface.
- **cat** – Directly captures what you type in the terminal. Type your contents, then press **Ctrl+D** on a new line to save and close the stream.
- **printf** – Evaluates formatting strings (like **\n** for new lines) and pushes the generated output straight into a file.
- **mv** – Reallocates an existing resource to a new name or destination path, effectively creating the file at the target location.

bash

```
touch newfile.txt
```

bash

```
printf "First Line\nSecond Line\n" > newfile.txt
```

bash

```
cat > newfile.txt
```

bash

```
mv existing_backup.txt new_report.txt
```

💻 Text Editors (Interactive)

- **vi/vim** – Opens the interactive Vim interfaces. Press **i** to enter Insert Mode, type your text, press **Esc**, and type **:wq!** to save and exit, **dd** – delete the line.
- **nano** – Opens a clean terminal interface. Type text directly, press **Ctrl+O** followed by **Enter** to save, and **Ctrl+X** to exit.
- **gedit** – Opens a standalone graphical window. The **&** symbol runs it in the background, keeping your terminal free for other tasks.

bash

```
vim filename.txt
```

bash

```
nano filename.txt
```

bash

```
gedit filename.txt &
```

Categories of File Creation Methods: Trade-off Analysis



⚖️ Editors vs. Command-Line Tools: Pros & Cons

Text Editors (Interactive)

- 👍 **Pros:** Excellent for making precise, live changes to files; visually intuitive with real-time text navigation.
- 👎 **Cons:** Requires user intervention to exit; completely halts script automation because it expects human interaction.

Command-Line Tools (Non-Interactive)

- 👍 **Pros:** Ideal for automation and shell scripts; creates files instantly without freezing or opening a new terminal window.
- 👎 **Cons:** Impractical for complex multi-line text modification; requires using terminal shortcuts or heavy syntax patterns to edit existing content.

Categorisation of File Viewing Methods

To view file contents in Linux, commands are separated by **how they process data** (streaming the whole file vs showing specific parts) and **how you interact with them** (scrolling step-by-step vs static output)

- **Whole-File Streaming Tools:** These print the entire file content directly to your terminal screen all at once.
  - cat (Standard forward view)
  - tac (Reverse line-by-line view)
  - rev (Reverse character-by-character view)
- **Partial-File Filtering Tools**
- **Interactive Pagers**

Whole-File Streaming Tools:

These print the entire file content directly to your terminal screen all at once.

- cat (Standard forward view)
- tac (Reverse line-by-line view)
- rev (Reverse character-by-character view)

cat - Streams and displays the entire contents of a file sequentially from the first line to the last.

tac - Streams the complete file in reverse order, displaying the last line first and working upward.

rev - Reverses the character order of every individual line horizontally from right to left.

Partial-File Filtering Tools

These target and isolate specific sections of a file rather than dumping whole text.

- head (Top slice view)
- tail (Bottom slice view / real-time monitoring view)

head - Extracts and prints a specified number of initial lines from the very top of a document.

tail - Extracts final lines from the bottom of a file, or dynamically tracks incoming log data in real-time.

Interactive Pagers

These open an interactive terminal layer that lets you comfortably scroll and read large documents page-by-page.

- less (Modern, bidirectional navigation pager)
- more (Legacy, forward-only navigation pager)

less - Opens a highly efficient interactive screen allowing multi-directional page scrolling and pattern searching.

more - Opens a basic interactive screen that pages through text sequentially in a forward direction only.

💻 Command Examples

Cat

```bash
cat standard_log.txt
```

Tac

```bash
tac stack_trace.log
```

rev

```bash
rev secret_key.txt
```

head

```bash
head -n 15 configurations.conf
```

tail

```bash
tail -f live_traffic.log
```

less

```bash
less massive_database.csv
```

more

```bash
more readable_guide.txt
```

Categorisation of Hidden Methods

We can segregate hidden file operations into two categories based on whether you are building them from scratch or hiding something that already exists.

- **Direct Hidden Creation**
- **Post-Creation Hiding (Renaming)**

Direct Hidden Creation

Creating a brand new file or folder that is instantly hidden from standard view.

- touch .filename (Hidden file creation)
- mkdir .dirname (Hidden directory creation)

touch .file – Creates an empty, hidden text file by prefixing the filename with a single dot.

bash

```
touch .secret_config.txt
```

mkdir .dir – Provisons a hidden directory folder container that disappears from standard file listings.

bash

```
mkdir .hidden_vault
```

Post-Creation Hiding (Renaming)

Taking an unhidden file or directory and converting it into a hidden one.

- mv filename .filename (Hiding an existing file)
- mv dirname/ .dirname/ (Hiding an existing directory)

mv file .file – Converts a visible file into a hidden file by rewriting its path string to start with a dot.

bash

```
mv public_draft.txt .hidden_draft.txt
```

Categorisation of Copying Methods

We can segregate copying methods based on the target type and safety constraints:

- **Standard File Copying**
  - cp source destination (Standard single file copy)
  - cp file1 file2 file3 target_directory/ (Multi-file copy to a folder)
- **Directory (Folder) Copying**
  - cp –r source_dir/ target_dir/ (Recursive directory copy)
  - cp source_dir/ target_dir/ (Empty directory copy)
- **Safety & Verification Copying**
  - cp –i source destination (Interactive mode/ overwrite prompt)
  - cp –v source destination (Verbose mode/ live feedback)

💻 Command Examples

Standard Single File Copy - Point out that **report.txt** still exists in its original state. A brand new clone named **backup_report.txt** has been created side-by-side with it.

bash

```
cp report.txt backup_report.txt
```

Copying Multiple Files into a Directory – When copying multiple items at once, the **very last argument** must always be an existing directory.

bash

```
cp photo1.jpg photo2.jpg photo3.jpg media_folder/
```

💻 Command Examples

Copying an Entire Directory (-r) - Crucial troubleshooting point! If you try to copy a directory without the -r (recursive) flag, Linux will throw an error saying "omitting directory". The -r tells Linux to dive deep inside the folder and copy everything sequentially.

```bash
cp -r Documents/ External_Backup/
```

Safe Overwriting (-i) – The **–i** stands for Interactive. If **system.conf** already exists, Linux will stop and ask: **overwrite/etc/system.conf? (y/n).** This saves administrators from catastrophic typos.

```bash
cp -i update.conf /etc/system.conf
```

Seeing the Action (-v) – The –v stands for **Verbose**. Instead of executing silently, the terminal prints 'document.pdf' -> 'Archival/document.pdf', which provides instant visual validation.

```bash
cp -v document.pdf Archival/
```

Categorisation of Moving and Renaming Methods

We can segregate directory operations based on **where** the folder is going:

- In-Place Remaining
- Location Relocation (Moving)
- Combined Move & Rename
- Safety Interventions

In-Place Remaining

Keeping the directory in its current folder location but changing its visual identity string.

- mv old_name/ new_name/ (Renaming an isolated directory)
- mv old_dir new_dir: Changes a directory’s name instantly while leaving its internal files completely untouched.

1. **Simple Directory Renaming** - Linux doesn't re-copy the files inside; it simply rewrites the folder's name pointer in the filesystem index.

```bash
mv project_draft/ project_final/
```

Location Relocation (Moving)

Relocating a folder intact to a completely different branch of the file system tree.

- mv source_dir/ existing_parent_dir/ (Moving a directory into another folder)

1. **Move a Directory Into Another Folder** - If a directory named assets already exists, this command drops the entire images folder inside it (assets/images/). If assets does not exist, it defaults back to method 1 and renames images to assets.

```bash
mv images/ assets/
```

Combined Move & Rename

Moving a folder to a distant location and changing its name at the exact same time.

- mv old_location/dir1 /new_location/dir2 (Simultaneous migration and rename)

Combined Move and Rename – This takes the folder **movie** out of the downloads directory, teleports it over to the media drive, and renames the folder container to **cinema** all in one single step

bash

```
mv downloads/movie/ /media/entertainment/cinema/
```

Safety Interventions

Overriding or securing the move to prevent unintended directory merging or overwriting.

- mv -i source/ target/ (Interactive safeguard mode)

Interactive Guardrail Mode (-i) – The **–i** stands for **Interactive.** If you accidentally type a directory move that might overwrite or destructively merge critical production data, this flag stops the execution and forces a y/n confirmation prompt.

```bash
mv -i staging/ production/
```